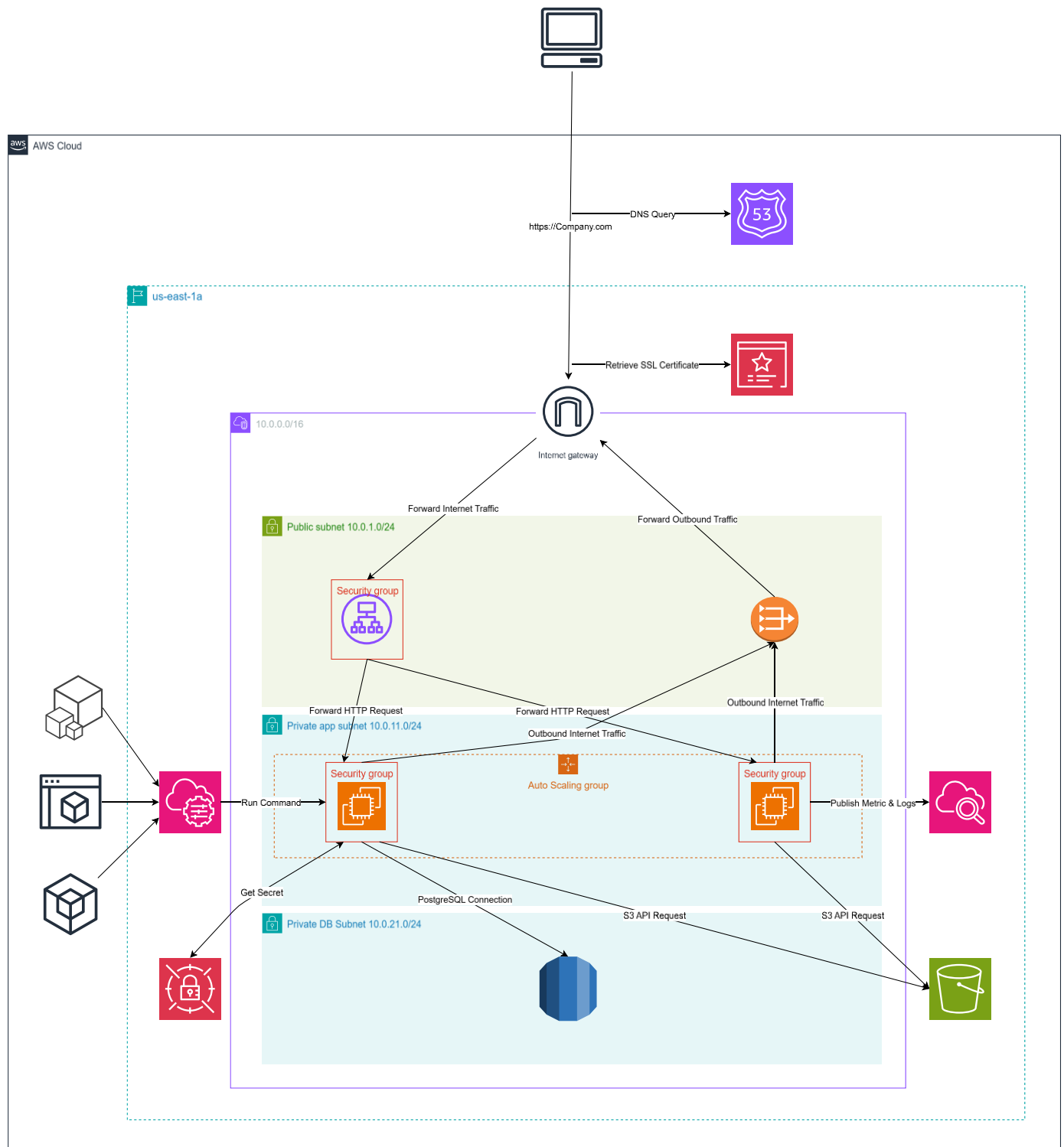


Enterprise Web App Platform

Table of Content

Phase 1 — Networking Foundation	4
Phase 2 — Security	9
Phase 3 — Compute	14
Phase 4 — Traffic Management	17
Phase 5 — Database	18
Phase 6 — Storage	19
Phase 7 — Domain & SSL	21
Phase 8 — Secrets Management	22
Phase 9 — Monitoring	25
Phase 10 — Systems Management	28



Step 0.

```
python3 -m venv ministackEnv
source ministackEnv/bin/activate
ministack
open new terminal tab
source ministackEnv/bin/activate
```

Step 1. Env Var

```
export AWS_ACCESS_KEY_ID=test
export AWS_SECRET_ACCESS_KEY=test
export AWS_DEFAULT_REGION=us-east-1
```

Phase 1 — Networking Foundation

Step 2. Membuat VPC dengan CIDR 10.0.0.0/16

```
aws --endpoint-url=http://localhost:4566 ec2 create-vpc \
    --cidr-block 10.0.0.0/16 \
    --tag-specifications
'ResourceType=vpc,Tags=[{Key=Name,Value=enterprise-vpc}]'
```

Notes : cek kalo udah dibuat

```
aws --endpoint-url=http://localhost:4566 ec2 describe-vpcs
```

Step 3. Create Subnet

1. Ambil VPC ID terlebih dahulu berdasarkan Tag dan simpan di RAM

```
VPC_ID=$(aws ec2 describe-vpcs \
    --endpoint-url=http://localhost:4566 \
    --filters "Name=tag:Name,Values=enterprise-vpc" \
    --query "Vpcs[*].VpcId" \
    --output text)
```

2. Membuat Public Subnet (Menggunakan variabel \$VPC_ID yang sudah siap)

```
aws --endpoint-url=http://localhost:4566 ec2 create-subnet \
    --vpc-id $VPC_ID \
```

```

--cidr-block 10.0.1.0/24 \
--availability-zone us-east-1a \
--tag-specifications
'ResourceType=subnet,Tags=[{Key=Name,Value=public-subnet}]'
# 3. Membuat Private App Subnet
aws --endpoint-url=http://localhost:4566 ec2 create-subnet \
--vpc-id $VPC_ID \
--cidr-block 10.0.11.0/24 \
--availability-zone us-east-1a \
--tag-specifications
'ResourceType=subnet,Tags=[{Key=Name,Value=private-app-subnet}]'

```

4. Membuat Private DB Subnet

```

aws --endpoint-url=http://localhost:4566 ec2 create-subnet \
--vpc-id $VPC_ID \
--cidr-block 10.0.21.0/24 \
--availability-zone us-east-1a \
--tag-specifications
'ResourceType=subnet,Tags=[{Key=Name,Value=private-db-subnet}]'

```

Notes : cara lama → <VPC_ID> lihat di

Global (seluruh vpc masuk)

aws --endpoint-url=http://localhost:4566 ec2 describe-vpcs → describe seluruh vpc yg ada di arsitektur aws kita

ATAU

aws --endpoint-url=http://localhost:4566 ec2 describe-vpc --query "Vpcs[*].VpcId" --output text → coba dong show vpcID dari seluruh VPC

Local (Khusus)

```

aws ec2 describe-vpcs \
--endpoint-url=http://localhost:4566 \
--filters "Name=tag:Name,Values=enterprise-vpc"

```

ATAU

```

aws ec2 describe-vpcs \
--endpoint-url=http://localhost:4566 \
--filters "Name=tag:Name,Values=enterprise-vpc" \
--query "Vpcs[*].VpcId" \
--output text

```

Step 4. Create Internet Gateway

Create Internet Gateway

```
aws --endpoint-url=http://localhost:4566 ec2 create-internet-gateway \
    --tag-specifications 'ResourceType=internet-gateway,Tags=[{Key=Name,Value=enterprise-igw}]' \
```

IGW_ID biar cepet

```
IGW_ID=$(aws ec2 create-internet-gateway \
    --endpoint-url=http://localhost:4566 \
    --tag-specifications 'ResourceType=internet-gateway,Tags=[{Key=Name,Value=enterprise-igw}]' \
    --query 'InternetGateway.InternetGatewayId' \
    --output text)
```

Attach Internet Gateway ke VPC

```
aws --endpoint-url=http://localhost:4566 ec2 attach-internet-gateway \
    --internet-gateway-id $IGW_ID \
    --vpc-id $VPC_ID
```

Notes :

```
aws --endpoint-url=http://localhost:4566 ec2 describe-nat-gateways
```

Step 5. Create NAT Gateway

Allocate Elastic IP → untuk meminta ke aws ip public brapapun itu yang static

```
aws --endpoint-url=http://localhost:4566 ec2 allocate-address \
    --domain vpc
```

PUBLIC_SUBNET_ID dan ALLOCATION_ID ke variable

```
PUBLIC_SUBNET_ID=$(
aws --endpoint-url=http://localhost:4566 ec2 create-subnet \
    --vpc-id "$VPC_ID" \
    --cidr-block 10.0.1.0/24 \
    --query 'Subnet.SubnetId' \
    --output text
)

ALLOCATION_ID=$(
aws --endpoint-url=http://localhost:4566 ec2 allocate-address \
    --domain vpc \
```

```

        --query 'AllocationId' \
        --output text
    )
# Create NAT Gateway
aws --endpoint-url=http://localhost:4566 ec2 create-nat-gateway \
    --subnet-id $PUBLIC_SUBNET_ID \
    --allocation-id $ALLOCATION_ID \
    --tag-specifications
'ResourceType=natgateway,Tags=[{Key=Name,Value=enterprise-nat}]'

```

Notes :

```
aws --endpoint-url=http://localhost:4566 ec2 describe-nat-gateways
```

Step 6. Create Route Tables

Menyiapkan seluruh variable biar cepet (PUBLIC_RT_ID,PRIVATE_RT_ID, NAT_GATEWAY_ID

```

PUBLIC_RT_ID=$(
aws --endpoint-url=http://localhost:4566 ec2 create-route-table \
    --vpc-id $VPC_ID \
    --query 'RouteTable.RouteTableId' \
    --output text
)

PRIVATE_RT_ID=$(
aws --endpoint-url=http://localhost:4566 ec2 create-route-table \
    --vpc-id $VPC_ID \
    --query 'RouteTable.RouteTableId' \
    --output text
)

NAT_GATEWAY_ID=$(
aws --endpoint-url=http://localhost:4566 ec2 create-nat-gateway \
    --subnet-id $PUBLIC_SUBNET_ID \
    --allocation-id $ALLOCATION_ID \
    --tag-specifications
'ResourceType=natgateway,Tags=[{Key=Name,Value=enterprise-nat}]' \
    --query 'NatGateway.NatGatewayId' \
    --output text
)

```

```

)

PRIVATE_APP_SUBNET_ID=$(
aws --endpoint-url=http://localhost:4566 ec2 create-subnet \
    --vpc-id $VPC_ID \
    --cidr-block 10.0.11.0/24 \
    --query 'Subnet.SubnetId' \
    --output text
)

PRIVATE_DB_SUBNET_ID=$(
aws --endpoint-url=http://localhost:4566 ec2 create-subnet \
    --vpc-id $VPC_ID \
    --cidr-block 10.0.21.0/24 \
    --query 'Subnet.SubnetId' \
    --output text
)

# Public Route Table

aws --endpoint-url=http://localhost:4566 ec2 create-route-table \
    --vpc-id $VPC_ID

# Tambahkan Route ke Internet Gateway

aws --endpoint-url=http://localhost:4566 ec2 create-route \
    --route-table-id $PUBLIC_RT_ID \
    --destination-cidr-block 0.0.0.0/0 \
    --gateway-id $IGW_ID

# Associate ke Public Subnet

aws --endpoint-url=http://localhost:4566 ec2 associate-route-table \
    --subnet-id $PUBLIC_SUBNET_ID \
    --route-table-id $PUBLIC_RT_ID

# Private Route Table

aws --endpoint-url=http://localhost:4566 ec2 create-route-table \
    --vpc-id $VPC_ID

# Tambahkan Route ke NAT Gateway

aws --endpoint-url=http://localhost:4566 ec2 create-route \
    --route-table-id $PRIVATE_RT_ID \
    --destination-cidr-block 0.0.0.0/0 \

```



```

--nat-gateway-id $NAT_GATEWAY_ID

# Associate ke Private Subnet
aws --endpoint-url=http://localhost:4566 ec2 associate-route-table \
    --subnet-id $PRIVATE_APP_SUBNET_ID \
    --route-table-id $PRIVATE_RT_ID
aws --endpoint-url=http://localhost:4566 ec2 associate-route-table \
    --subnet-id $PRIVATE_DB_SUBNET_ID \
    --route-table-id $PRIVATE_RT_ID

```

Notes :

```

# 1. Cek Route Table
aws --endpoint-url=http://localhost:4566 ec2 describe-route-tables

# 2. Cek Association Route Table
aws --endpoint-url=http://localhost:4566 ec2 describe-route-tables --
query "RouteTables[*].Associations"

# 3. Cek Route
aws --endpoint-url=http://localhost:4566 ec2 describe-route-tables --
query "RouteTables[*].Routes"

```

Phase 2 — Security

Step 7. Security Group

```

# Create Security Group untuk ALB
SG_ALB_ID=$(
aws --endpoint-url=http://localhost:4566 ec2 create-security-group \
    --group-name sg-alb \
    --description "Security Group for ALB" \
    --vpc-id "$VPC_ID" \
    --query 'GroupId' \
    --output text
)

# Izinkan HTTPS dari Internet
aws --endpoint-url=http://localhost:4566 ec2 authorize-security-group-
ingress \
    --group-id "$SG_ALB_ID" \

```

```

        --protocol tcp \
        --port 443 \
        --cidr 0.0.0.0/0

# Create Security Group untuk EC2

SG_EC2_ID=$(
aws --endpoint-url=http://localhost:4566 ec2 create-security-group \
    --group-name sg-ec2 \
    --description "Security Group for EC2" \
    --vpc-id "$VPC_ID" \
    --query 'GroupId' \
    --output text
)

# Izinkan HTTP hanya dari ALB

aws --endpoint-url=http://localhost:4566 ec2 authorize-security-group-
ingress \
    --group-id "$SG_EC2_ID" \
    --protocol tcp \
    --port 80 \
    --source-group "$SG_ALB_ID"

# Create Security Group untuk RDS

SG_RDS_ID=$(
aws --endpoint-url=http://localhost:4566 ec2 create-security-group \
    --group-name sg-rds \
    --description "Security Group for RDS" \
    --vpc-id "$VPC_ID" \
    --query 'GroupId' \
    --output text
)

# Izinkan PostgreSQL hanya dari EC2

aws --endpoint-url=http://localhost:4566 ec2 authorize-security-group-
ingress \
    --group-id "$SG_RDS_ID" \
    --protocol tcp \
    --port 5432 \
    --source-group "$SG_EC2_ID"

```

Step 8. IAM

8.1 trust-policy.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "ec2.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

8.2 enterprise-ec2-policy.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadWriteApplicationBucket",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::enterprise-app-bucket",
        "arn:aws:s3:::enterprise-app-bucket/*"
      ]
    },
    {
      "Sid": "ReadDatabaseSecret",
```

```

        "Effect": "Allow",
        "Action": [
            "secretsmanager:GetSecretValue"
        ],
        "Resource": "*"
    },
    {
        "Sid": "CloudWatchLogs",
        "Effect": "Allow",
        "Action": [
            "logs:CreateLogGroup",
            "logs:CreateLogStream",
            "logs:PutLogEvents"
        ],
        "Resource": "*"
    }
]
}

# Create IAM Role
aws --endpoint-url=http://localhost:4566 iam create-role \
    --role-name EnterpriseEC2Role \
    --assume-role-policy-document file:///trust-policy.json

# Create Customer Managed Policy
aws --endpoint-url=http://localhost:4566 iam create-policy \
    --policy-name EnterpriseEC2Policy \
    --policy-document file:///enterprise-ec2-policy.json

# Attach Permission Policy into Role
aws --endpoint-url=http://localhost:4566 iam attach-role-policy \
    --role-name EnterpriseEC2Role \
    --policy-arn arn:aws:iam::000000000000:policy/EnterpriseEC2Policy

# Buat Instance Profile
aws --endpoint-url=http://localhost:4566 iam create-instance-profile \
    --instance-profile-name EnterpriseEC2Profile

# Tambahkan Role ke Instance Profile

```

```
aws --endpoint-url=http://localhost:4566 iam add-role-to-instance-  
profile \  
    --instance-profile-name EnterpriseEC2Profile \  
    --role-name EnterpriseEC2Role
```

Notes:

Validasi

1. Cek Role sudah dibuat

```
aws --endpoint-url=http://localhost:4566 iam get-role \  
    --role-name EnterpriseEC2Role
```

2. Cek Trust Policy

```
aws --endpoint-url=http://localhost:4566 iam get-role \  
    --role-name EnterpriseEC2Role
```

3. Cek Permission Policy sudah dibuat

```
aws --endpoint-url=http://localhost:4566 iam get-policy \  
    --policy-arn arn:aws:iam::000000000000:policy/EnterpriseEC2Policy
```

4. Cek isi Permission Policy

```
aws --endpoint-url=http://localhost:4566 iam get-policy \  
arn:aws:iam::000000000000:policy/EnterpriseEC2Policy
```

kalo keluarnya v1

```
aws --endpoint-url=http://localhost:4566 iam get-policy-version \  
    --policy-arn arn:aws:iam::000000000000:policy/EnterpriseEC2Policy  
\  
    --version-id v1
```

5. Cek Policy sudah di-attach

```
aws --endpoint-url=http://localhost:4566 iam list-attached-role-  
policies \  
    --role-name EnterpriseEC2Role
```

6. Cek Instance Profile

```
aws --endpoint-url=http://localhost:4566 iam get-instance-profile \  
    --instance-profile-name EnterpriseEC2Profile
```

7. Cek Role sudah masuk Instance Profile

Masih dari command tadi.

Cari

```
"Roles": [EnterpriseEC2Role]
```

Phase 3 — Compute

userdata.sh

```
#!/bin/bash

set -eux

# =====
# Update package
# =====

apt update

# =====
# Install Docker
# =====

apt install -y docker.io

systemctl enable docker
systemctl start docker

# =====
# Install CloudWatch Agent → nanti dibahas di phase 9. Monitoring
# =====

wget https://amazoncloudwatch-agent.s3.amazonaws.com/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb

dpkg -i amazon-cloudwatch-agent.deb

# =====
# Contoh konfigurasi sederhana
# =====

mkdir -p /opt/aws/amazon-cloudwatch-agent/etc
```

```
cat > /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json <<EOF
```

```
{
  "metrics": {
    "namespace": "EnterpriseWebApp",
    "append_dimensions": {
      "InstanceId": "\${aws:InstanceId}"
    },
    "metrics_collected": {
      "cpu": {
        "measurement": [
          "cpu_usage_idle",
          "cpu_usage_user",
          "cpu_usage_system"
        ],
        "totalcpu": true
      },
      "mem": {
        "measurement": [
          "mem_used_percent"
        ]
      },
      "disk": {
        "measurement": [
          "used_percent"
        ],
        "resources": [
          "*"
        ]
      }
    }
  }
}
```

EOF

```
# =====

# Start CloudWatch Agent → nanti dibahas di phase 9. Monitoring

# =====

systemctl enable amazon-cloudwatch-agent

/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
-a fetch-config \
-m ec2 \
-c file:/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json \
-s

# =====

# Run Application

# =====

docker run -d \
  --name web \
  -p 80:80 \
  nginx
```

Step 9. Launch Template → blueprint yang digunakan Auto Scaling Group untuk membuat EC2 baru.

Create Launch Template

```
aws --endpoint-url=http://localhost:4566 ec2 create-launch-template \
  --launch-template-name enterprise-launch-template \
  --launch-template-data "{
    \"ImageId\": \"ami-xxxxxxx\",
    \"InstanceType\": \"t3.micro\",
    \"SecurityGroupIds\": [\"$SG_EC2_ID\"],
    \"IamInstanceProfile\": {
      \"Name\": \"EnterpriseEC2Profile\"
    },
    \"UserData\": \"$(base64 -w0 userdata.sh)\"
  }
```



```
}"
```

Step 10. Auto Scaling Group

Create Auto Scaling Group

```
aws --endpoint-url=http://localhost:4566 autoscaling create-auto-  
scaling-group \  
    --auto-scaling-group-name enterprise-asg \  
    --launch-template LaunchTemplateName=enterprise-launch-  
template,Version='$Latest' \  
    --min-size 2 \  
    --desired-capacity 2 \  
    --max-size 10 \  
    --vpc-zone-identifier "$PRIVATE_APP_SUBNET_ID"
```

Scaling Policy (diatas 70% cpu, bikin baru)

```
aws --endpoint-url=http://localhost:4566 autoscaling put-scaling-  
policy \  
    --auto-scaling-group-name enterprise-asg \  
    --policy-name cpu-scale-out \  
    --policy-type TargetTrackingScaling \  
    --target-tracking-configuration '{  
        "PredefinedMetricSpecification":{  
            "PredefinedMetricType":"ASGAverageCPUUtilization"  
        },  
        "TargetValue":70  
    }'
```

Phase 4 — Traffic Management

Step 11 — Target Group

Untuk menyimpan daftar server, Health Check, Regis dan Deregis, Load Balancing

```
TG_ARN=$(  
aws --endpoint-url=http://localhost:4566 elbv2 create-target-group \  
    --name enterprise-target-group \  
    --protocol HTTP \  
    --port 80 \  
)
```

```

--vpc-id "$VPC_ID" \
--target-type instance \
--health-check-path "/" \
--query 'TargetGroups[0].TargetGroupArn' \
--output text
)

# validasi Target Group
aws --endpoint-url=http://localhost:4566 elbv2 describe-target-groups

```

Step 12 — Application Load Balancer (ALB)

create ALB

```

ALB_ARN=$(
aws --endpoint-url=http://localhost:4566 elbv2 create-load-balancer \
--name enterprise-alb \
--subnets "$PUBLIC_SUBNET_ID" \
--security-groups "$SG_ALB_ID" \
--scheme internet-facing \
--type application \
--query 'LoadBalancers[0].LoadBalancerArn' \
--output text
)

```

validasi ALB

```

aws --endpoint-url=http://localhost:4566 elbv2 describe-load-balancers

```

Create Listener, menentukan bagaimana ALB menerima request dan ke mana request tersebut diteruskan.

```

aws --endpoint-url=http://localhost:4566 elbv2 create-listener \
--load-balancer-arn "$ALB_ARN" \
--protocol HTTP \
--port 80 \
--default-actions Type=forward,TargetGroupArn="$TG_ARN"

```

Validasi Listener

```

aws --endpoint-url=http://localhost:4566 elbv2 describe-listeners

```

Phase 5 — Database

Step 13. Amazon RDS

13.1 Membuat DB Subnet Group

Di AWS production, DB Subnet Group **wajib berisi minimal 2 subnet di Availability Zone yang berbeda** agar mendukung Multi-AZ. Karena MiniStack/LocalStack bersifat simulasi, kita menggunakan satu subnet agar sesuai dengan arsitektur project.

```
aws --endpoint-url=http://localhost:4566 rds create-db-subnet-group \
    --db-subnet-group-name enterprise-db-subnet-group \
    --db-subnet-group-description "Enterprise DB Subnet Group" \
    --subnet-ids "$PRIVATE_DB_SUBNET_ID"
```

Membuat Amazon RDS PostgreSQL

```
aws --endpoint-url=http://localhost:4566 rds create-db-instance \
    --db-instance-identifier enterprise-postgres \
    --engine postgres \
    --db-instance-class db.t3.micro \
    --allocated-storage 20 \ #20gb
    --master-username postgres \
    --master-user-password Password123! \
    --vpc-security-group-ids "$SG_RDS_ID" \
    --db-subnet-group-name enterprise-db-subnet-group \
    --publicly-accessible false
```

Validasi DB

```
aws --endpoint-url=http://localhost:4566 rds describe-db-instances
```

Phase 6 — Storage

Step 14 — Amazon S3

Create bucket

```
aws --endpoint-url=http://localhost:4566 s3 mb s3://enterprise-storage
```

Melihat bucket

```
aws --endpoint-url=http://localhost:4566 s3 ls
```

Upload file (bisa local to s3 bisa sebaliknya, klo disini yg pertama)

```
aws --endpoint-url=http://localhost:4566 s3 cp logo.png
s3://enterprise-storage/logo.png
```

Melihat isi Bucket

```
aws --endpoint-url=http://localhost:4566 s3 ls s3://enterprise-storage
```

Download file

```
aws --endpoint-url=http://localhost:4566 s3 cp s3://enterprise-storage/logo.png .
```

Hapus file

```
aws --endpoint-url=http://localhost:4566 s3 rm s3://enterprise-storage/logo.png
```

Mengaktifkan versioning → Versioning memungkinkan setiap perubahan file disimpan sebagai versi baru sehingga file lama dapat dipulihkan.

```
aws --endpoint-url=http://localhost:4566 s3api put-bucket-versioning \
    --bucket enterprise-storage \
    --versioning-configuration Status=Enabled
```

Contoh lifecycle rule → File log lebih dari 30 hari dipindahkan ke storage yang lebih murah. File backup lebih dari 365 hari dihapus otomatis.

```
aws --endpoint-url=http://localhost:4566 s3api put-bucket-lifecycle-configuration \
    --bucket enterprise-storage \
    --lifecycle-configuration '{
        "Rules": [{
            "ID": "DeleteOldLogs",
            "Status": "Enabled",
            "Filter": {
                "Prefix": "logs/"
            },
            "Expiration": {
                "Days": 30
            }
        }]
    }'
```

Notes: Beberapa fitur seperti Lifecycle mungkin belum didukung sepenuhnya oleh MiniStack/LocalStack, tetapi konfigurasi ini tetap merepresentasikan cara implementasi di AWS asli.

Phase 7 — Domain & SSL

Step 15. Amazon Route53

15.1 Create Hosted Zone

```
HOSTED_ZONE_ID=$(  
aws --endpoint-url=http://localhost:4566 route53 create-hosted-zone \  
    --name company.com \  
    --caller-reference $(date +%s) \ # ID unik agar request tidak  
    dianggap duplikat.  
    --query 'HostedZone.Id' \  
    --output text  
)
```

15.2 record.json → Membuat Alias Record ke ALB

```
{  
  "Comment": "Alias ke ALB",  
  "Changes": [  
    {  
      "Action": "CREATE",  
      "ResourceRecordSet": {  
        "Name": "company.com",  
        "Type": "A",  
        "AliasTarget": {  
          "HostedZoneId": "ALB_HOSTED_ZONE_ID",  
          "DNSName": "ALB_DNS_NAME",  
          "EvaluateTargetHealth": false  
        }  
      }  
    }  
  ]  
}
```

15.3

```
aws --endpoint-url=http://localhost:4566 route53 change-resource-  
record-sets \  

```

```
--hosted-zone-id "$HOSTED_ZONE_ID" \  
--change-batch file://record.json
```

Step 16 — AWS Certificate Manager (ACM)

Request Certificate

```
aws --endpoint-url=http://localhost:4566 acm request-certificate \  
--domain-name company.com \  
--validation-method DNS
```

Melihat Certificate

```
aws --endpoint-url=http://localhost:4566 acm list-certificates
```

Menghubungkan ACM dengan ALB

```
aws --endpoint-url=http://localhost:4566 elbv2 create-listener \  
--load-balancer-arn "$ALB_ARN" \  
--protocol HTTPS \  
--port 443 \  
--certificates CertificateArn="$CERTIFICATE_ARN" \  
--default-actions Type=forward,TargetGroupArn="$TG_ARN"
```

Notes : Perlu diketahui bahwa Route53 dan ACM sulit atau tidak sepenuhnya dapat disimulasikan di MiniStack/LocalStack, terutama untuk:

- DNS publik yang benar-benar dapat di-resolve dari Internet.
- Validasi domain (DNS Validation) pada ACM.
- Sertifikat SSL/TLS yang dipercaya browser.

Karena itu, pada project ini **Route53 dan ACM lebih berfungsi sebagai pembelajaran konsep dan AWS CLI**, sedangkan pada deployment di AWS asli, keduanya akan bekerja penuh untuk memberikan domain publik (company.com) dan koneksi HTTPS yang valid.

Phase 8 — Secrets Management

Step 17. AWS Secrets Manager

Membuat Secret

```
aws --endpoint-url=http://localhost:4566 secretsmanager create-secret \  
\
```

```

--name enterprise/database \

--description "PostgreSQL Database Credential" \

--secret-string '{
    "username": "postgres",
    "password": "Password123!",
    "host": "enterprise-postgres",
    "port": "5432",
    "database": "enterprise_db"
}'

# Melihat secret

aws --endpoint-url=http://localhost:4566 secretsmanager list-secrets

# Mengambil secret

aws --endpoint-url=http://localhost:4566 secretsmanager get-secret-
value \

    --secret-id enterprise/database

# Contoh Mengambil Secret dari Aplikasi (Python)

import boto3

import json

client = boto3.client("secretsmanager")

secret = client.get_secret_value(
    SecretId="enterprise/database"
)

credential = json.loads(secret["SecretString"])

print(credential["username"])
print(credential["password"])

```

Notes :

- Bagaimana EC2 Mengakses Secret?

Di Phase 2 kita sudah membuat

IAM Role

↓

EnterpriseEC2Role

Lalu kita attach ke EC2.

Sekarang flow-nya menjadi

EC2

↓

IAM Role

↓

Secrets Manager

↓

Database Password

Kalau Role tidak memiliki permission

↓

AWS akan mengembalikan

AccessDeniedException

- Permission IAM yang Dibutuhkan

Contoh policy minimum

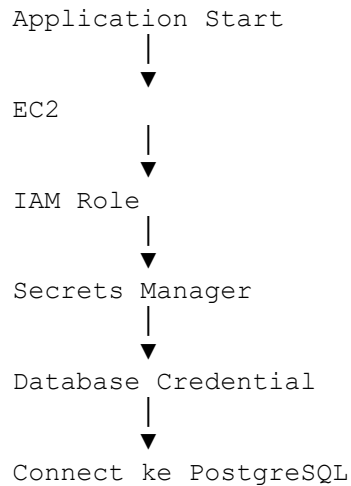
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "secretsmanager:GetSecretValue"
      ],
      "Resource": "*"
    }
  ]
}
```

Lalu attach policy tersebut ke

EnterpriseEC2Role

Sehingga hanya EC2 yang boleh membaca secret.

- Flow Lengkap



Pertanyaan selanjutnya kira kira buat RDS dulu trus bikin secret lalu disimpan di secret manager apa gmn?

Jawaban :

1. kalo di lab kita

1. Generate password random
2. Create RDS menggunakan password tersebut
3. Simpan credential ke Secret Manager
4. Aplikasi mengambil credential dari Secret Manager

2. best use case production

AWS punya fitur: **Manage master credentials in AWS Secrets Manager**

Kalau ini dicentang, AWS melakukan semuanya otomatis.

1. generate password random
2. buat RDS
3. simpan ke Secret Manager

Password bahkan kita tidak pernah lihat. Ini jauh lebih aman.

Phase 9 — Monitoring

Step 18 . Amazon CloudWatch

Yang Dimonitor

EC2

- CPU Utilization
- Network In
- Network Out
- Disk Read
- Disk Write
- Status Check

ALB

- Request Count
- Target Response Time (Latency)
- HTTP 4xx
- HTTP 5xx
- Healthy Host Count

RDS

- CPU
- Database Connection
- Free Storage
- Read IOPS
- Write IOPS

Application

- Error Log
- Access Log
- Exception
- Response Time

18.1 Membuat CloudWatch Alarm

```
aws --endpoint-url=http://localhost:4566 cloudwatch put-metric-alarm \  
  --alarm-name HighCPUAlarm \  
  --metric-name CPUUtilization \  
  --namespace AWS/EC2 \  
  --statistic Average \  
  --period 300 \  
  --threshold 80 \  
  --comparison-operator GreaterThanThreshold \  

```

```
--evaluation-periods 2 \
```

```
--dimensions Name=AutoScalingGroupName,Value=enterprise-asg
```

18.2.1 Menghubungkan Alarm ke Amazon SNS

Pertama buat Topic SNS.

```
aws --endpoint-url=http://localhost:4566 sns create-topic \
```

```
--name enterprise-alert
```

18.2.2 Subscribe email administrator.

```
aws --endpoint-url=http://localhost:4566 sns subscribe \
```

```
--topic-arn "$SNS_TOPIC_ARN" \
```

```
--protocol email \
```

```
--notification-endpoint admin@company.com
```

18.2.3 Kemudian alarm dikonfigurasi agar mengirim notifikasi ke SNS.

```
aws --endpoint-url=http://localhost:4566 cloudwatch put-metric-alarm \
```

```
--alarm-name HighCPUAlarm \
```

```
--metric-name CPUUtilization \
```

```
--namespace AWS/EC2 \
```

```
--statistic Average \
```

```
--period 300 \
```

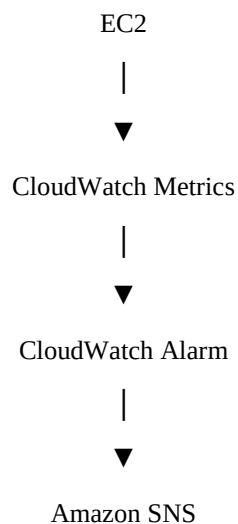
```
--threshold 80 \
```

```
--comparison-operator GreaterThanThreshold \
```

```
--evaluation-periods 2 \
```

```
--alarm-actions "$SNS_TOPIC_ARN"
```

Flow lengkapnya menjadi:



|

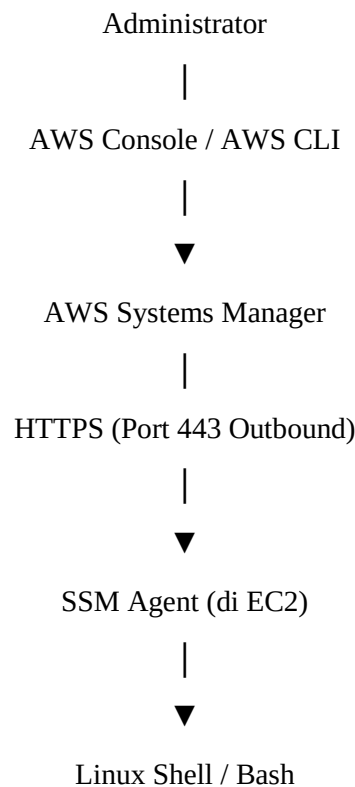


Email Administrator

Phase 10 — Systems Management

Pada phase terakhir ini kita akan mengelola EC2 menggunakan AWS Systems Manager (SSM). Sebelumnya administrator biasanya melakukan remote ke server menggunakan SSH melalui port 22 dan Public IP. Dengan SSM, administrator dapat membuka terminal langsung dari AWS Console atau AWS CLI tanpa membuka port SSH sama sekali, sehingga arsitektur menjadi lebih aman dan sesuai best practice modern.

Arsitektur setelah Phase 10:



Step 19. AWS Systems Manager (SSM)

Membuat IAM Policy untuk SSM Agar EC2 boleh terhubung ke Systems Manager, attach policy berikut ke IAM Role EC2.

```
aws --endpoint-url=http://localhost:4566 iam attach-role-policy \
    --role-name EnterpriseEC2Role \
    --policy-arn arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore
```

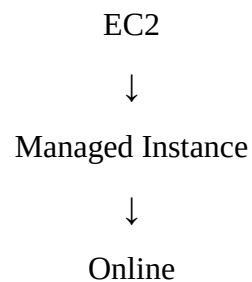
Install SSM Agent Pada Amazon Linux biasanya sudah terpasang. Kalau belum (Ubuntu):

```
sudo snap install amazon-ssm-agent --classic  
sudo systemctl enable snap.amazon-ssm-agent.amazon-ssm-agent.service  
sudo systemctl start snap.amazon-ssm-agent.amazon-ssm-agent.service
```

Melihat Managed Instance

```
aws --endpoint-url=http://localhost:4566 ssm describe-instance-  
information
```

Jika berhasil



Membuka Terminal Menggunakan AWS CLI Maka terminal Linux akan terbuka tanpa SSH :

```
aws --endpoint-url=http://localhost:4566 ssm start-session \  
--target <INSTANCE_ID>
```

Maka terminal Linux akan terbuka tanpa SSH.

Hubungan dengan Arsitektur Project

Setelah Phase 10 selesai, arsitektur kita menjadi:

